model.py

Credit Card Fraud Detection — Training & Evaluation

■ Overview

This script trains a **Random Forest Classifier** to detect fraudulent credit card transactions. The dataset is highly imbalanced (far more legitimate transactions than fraudulent ones), so **SMOTE** (Synthetic Minority Oversampling Technique) is applied to the training data before fitting the model. Two evaluation plots are also generated and saved.

■ Dependencies

- **pandas, numpy** — Data loading and numerical operations
- **scikit-learn** — train_test_split, RandomForestClassifier, metrics
- **imbalanced-learn** — SMOTE for handling class imbalance (optional — skipped gracefully if missing)
- **matplotlib, seaborn** — Plotting confusion matrix and precision-recall curve
- **os** — Checks for the presence of the CSV data file

■ Key Settings

DATA_FILE	creditcard.csv — must exist before running
Test size	20% of data held out for evaluation
Random state	42 (reproducible splits and model)
n_estimators	50 trees in the Random Forest
n_jobs	-1 (use all CPU cores for faster training)

■■ Pipeline — Step by Step

Step 1 — Load data

load_data() reads creditcard.csv with pandas. Raises FileNotFoundError if the file is missing.

Step 2 — Clean

Drops any rows with NaN values to ensure clean input.

Step 3 — Split features & label

X = all columns except 'Class'; y = 'Class' (0 = legitimate, 1 = fraud).

Step 4 — Train/test split

80% training, 20% test. stratify=y preserves the fraud ratio in both splits.

Step 5 — SMOTE oversampling

Synthetically generates minority (fraud) samples in the training set so the model sees balanced classes. Falls back gracefully if imbalanced-learn is not installed.

Step 6 — Train Random Forest

Fits RandomForestClassifier(n_estimators=50) on the SMOTE-resampled training data.

Step 7 — Evaluate

Predicts on the test set and prints a full classification report (precision, recall, F1 per class).

Step 8 — Confusion Matrix

Heatmap saved as confusion_matrix.png showing TP, FP, TN, FN counts.

Step 9 — Precision-Recall Curve

Plots the tradeoff between precision and recall at different thresholds. PR-AUC score is shown in the legend. Saved as precision_recall_curve.png.

■ Output Files

- **confusion_matrix.png** — heatmap of prediction results
- **precision_recall_curve.png** — PR curve with AUC score

■■ How to Run

Make sure **creditcard.csv** is in the same directory, then:

```
python lithish/model.py
```